

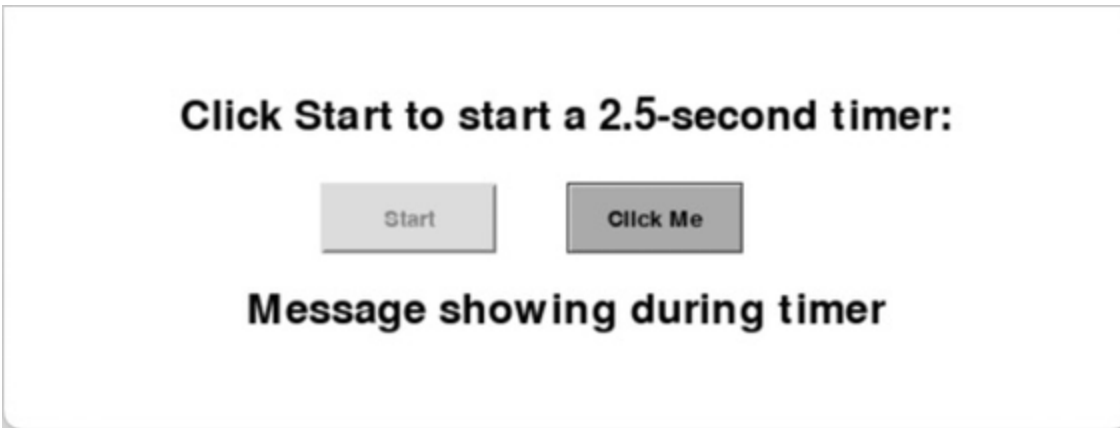


Figure 13-2: The message displayed while the timer is running

For two and a half seconds, the Start button becomes disabled and a message is displayed below the buttons. When the time expires, the message goes away and the Start button is re-enabled. Independent of the timer running, anything else that the user wants to do in the program still needs to be responsive. In this example, clicking Click Me prints a message to the shell window, whether the timer is running or not.

Three Approaches for Implementing Timers

In this section, I'll discuss three different approaches to implementing timers: counting frames, generating a pygame event, and checking for elapsed time. To make these concepts clear, the following code examples are built directly in the main loop.

Counting Frames

A straightforward approach to creating a timer is to count the number of frames that go by. One frame is the same as one loop iteration. If you know the frame rate of a program, you can calculate how long to wait by multiplying the time to wait by the frame rate. The following code shows the key parts of the implementation: